

Database Principles

CSC3400

CHAPTER 6

DATA MANIPULATION

LANGUAGE

Learning Objectives

- Define key terms: join, equijoin, self-join, nonequijoin, natural join, outer join, set operators and subquery.
- Applying the DML command to insert, update and delete data from table.
- Write single and multiple table queries using SQL commands
- Write queries using subqueries.

Data Manipulation Language (DML)

Used to create, modify and retrieve data

- ▣ Insert
- ▣ Select
- ▣ Update
- ▣ Delete

DML – Adding Data

Use the Insert keyword and specify:

- ▣ table name
- ▣ field names - optional
- ▣ values for each field

```
INSERT INTO customer VALUES ('Teplow', 'MA', 23445.67);
```

OR

```
INSERT INTO customer (last_name, state_cd, sales)  
VALUES ('Teplow' , 'MA', 23445.67);
```

DML – Update Data

Use the Update and Set keywords and specify:

- ▣ table name
- ▣ field name(s)
- ▣ where clause (optional)

```
UPDATE inventory  
SET price = price*1.05;
```

```
UPDATE inventory  
SET price = price*1.05  
WHERE product_id = 'P103';
```

DML – Remove Data

Use the Delete From keywords and specify:

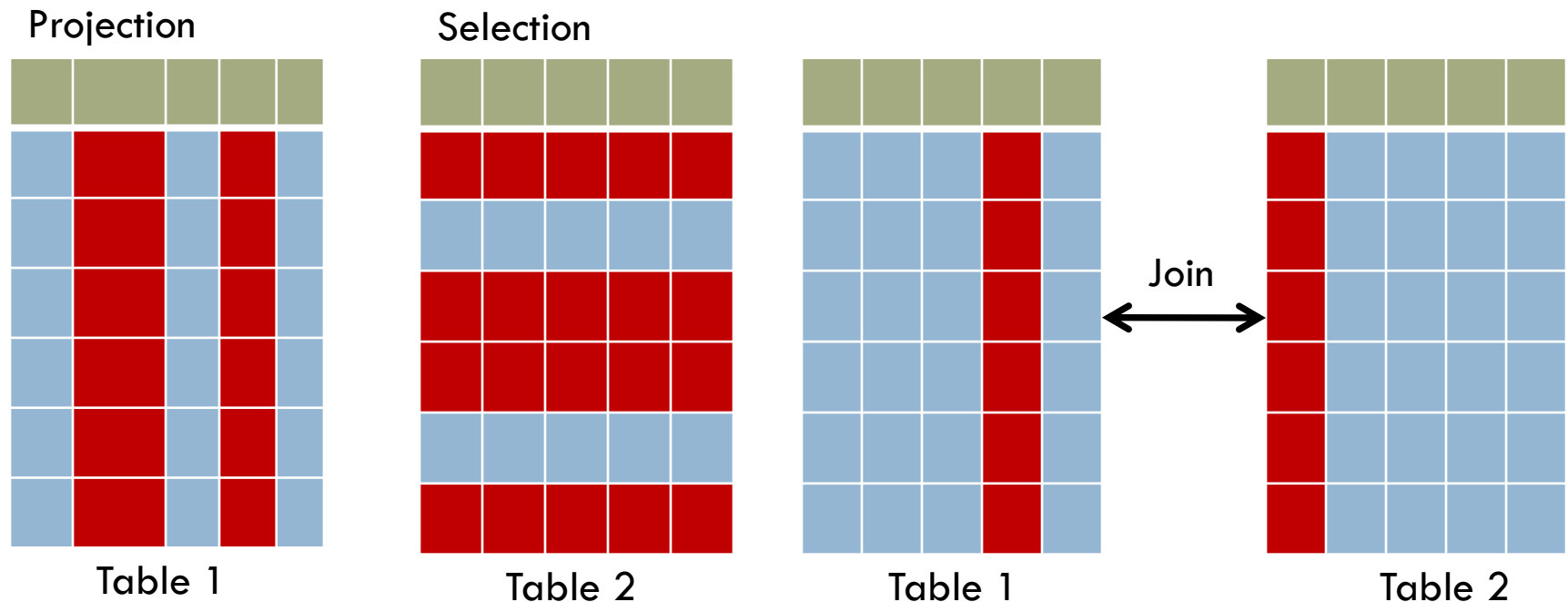
- ▣ table name
- ▣ where clause (optional)

```
DELETE FROM customer;
```

```
DELETE FROM customer  
WHERE sales < 10000;
```

DML – Select Data

- Select statement retrieves information from the databases.
- Capabilities of SQL Select Statements:



Basic Select Statement

- ❑ SELECT identifies the columns to be displayed
- ❑ FROM identifies the table containing those columns.

```
SELECT * | {[DISTINCT] column_name | expression [alias],.....}  
FROM table_name;
```

- ❑ DISTINCT suppresses duplicates
- ❑ * select all columns
- ❑ alias gives the selected columns different headings
- ❑ FROM identifies the table containing those columns.

Expressions

- Create expression with number and date by using arithmetic operators (i.e. $+$, $-$, $*$, $/$).
- If an expression contains more than one operator, $*$ and $/$ are evaluated first. If operators in an expression are of the same priority, evaluation is done from left to right.

```
SELECT last_name, salary, salary*12  
FROM employee;
```

LAST_NAME	SALARY	SALARY*12
Siti Razilah	1000	12000
Musa Ahmad	1200	14400
Martin Luke	1500	18000

Expressions ..cont.

```
SELECT last_name, salary, 12*salary+100  
FROM employee;
```

LAST_NAME	SALARY	SALARY*12
Siti Razilah	1000	12100
Musa Ahmad	1200	14500
Martin Luke	1500	18100

```
SELECT last_name, salary, 12*(salary+100)  
FROM employee;
```

LAST_NAME	SALARY	SALARY*12
Siti Razilah	1000	13200
Musa Ahmad	1200	15600
Martin Luke	1500	19200

Defining a Null Value

- ❑ Null is a value that is unavailable, unassigned, unknown, or inapplicable.
- ❑ Null is not the same as zero or a blank space.

```
SELECT last_name, salary, commission  
FROM employee;
```

LAST_NAME	SALARY	COMMISSION
Siti Razilah	1000	(null)
Musa Ahmad	1200	0.2
Martin Luke	1500	0.6
Ammar Faizul	1800	0.3
Safiyah Shahizan	1230	(null)
Taylor Duff	2500	0.8

Defining a Null Value ..cont.

```
SELECT last_name, salary, commission, salary*commission  
FROM employee;
```

LAST_NAME	SALARY	COMMISSION	SALARY*COMMISSION
Siti Razilah	1000	(null)	(null)
Musa Ahmad	1200	0.2	240
Martin Luke	1500	0.6	900
Ammar Faizul	1800	0.3	540
Safiyah Shahizan	1230	(null)	(null)
Taylor Duff	2500	0.8	2000

Column Heading Default

- Uppercase, however can be override the column heading with heading display with an alias.
- A column alias:
 - ▣ Is useful with calculations
 - ▣ Immediately follows the column name (There can be also the optional AS keyword between column name and the alias.)
 - ▣ Requires double quotation marks if it contains spaces or special characters, or if it is case-sensitive.
- Example:

```
SELECT last_name as name, nokp as identityno  
FROM employee;
```

NAME	IDENTITYNO
Siti Razilah	790120-14-6222
Musa Ahmad	800828-10-6323
Martin Luke	870401-12-6087

Column Heading Default *..cont.*

```
SELECT last_name as "NaMe", nokp as "identity no"  
FROM employee;
```

NaMe	Identity no
Siti Razilah	790120-14-6222
Musa Ahmad	800828-10-6323
Martin Luke	870401-12-6087

```
SELECT last_name as "**Name**", nokp as "#identityno"  
FROM employee;
```

Name	#identityno
Siti Razilah	790120-14-6222
Musa Ahmad	800828-10-6323
Martin Luke	870401-12-6087

Concatenation Operator

- Links columns or character strings to other columns
- Is represented by two vertical bars (||)
- Create a resultant column that is a character expression

```
SELECT last_name || first_name as "employeeName"  
FROM employee;
```

- If a null value with a character string, the result is a character string.

e.g: last_name || null → last_name

employeeName
Siti Razilah
Musa Ahmad
Martin Luke
Ammar Faizul
Safiyah Shahizan
Taylor Duff

Using Literal Character Strings

- ❑ Literal is a character, a number, or a date that is included in the select statement.
- ❑ It is not a column name or a column alias.
- ❑ Date and character literal values must be enclosed within single quotation marks; number literals need not be enclosed in a similar manner;
- ❑ Each character string is output once for each row returned.

Using Literal Character Strings ..cont.

```
SELECT last_name || 's 1 month salary =' || salary Monthly  
FROM employee;
```

MONTHLY
Siti Razilah :1 month salary = 1000
Musa Ahmad: 1 month salary = 1200
Martin Luke : 1 month salary = 1500
Ammar Faizul: 1 month salary = 1800
Safiyah Shahizan: 1 month salary = 1230
Taylor Duff: 1 month salary = 2500

Using Literal Character Strings ..cont.

```
SELECT last_name || '''s 1 month salary =' || salary Monthly  
FROM employee;
```

MONTHLY

Siti Razilah 's 1 month salary = 1000

Musa Ahmad's 1 month salary = 1200

Martin Luke 's 1 month salary = 1500

Ammar Faizul 's 1 month salary = 1800

Safiyah Shahizan's 1 month salary = 1230

Taylor Duff 's 1 month salary = 2500

Or you can use Quote (**q**) Operator
[], < >, { }, () are allowed



```
SELECT last_name || q['s 1 month salary ='] || salary Monthly  
FROM employee;
```

Duplicate Rows

- The default display of queries is all rows, including duplicate rows.

```
SELECT department_id  
FROM employee;
```

DEPARTMENT_ID
10
20
30
20
10
20

```
SELECT DISTINCT department_id  
FROM employee;
```

DEPARTMENT_ID
10
20
30

Note: If you specify multiple columns after the DISTINCT qualifier, the result is every distinct combination of the columns.

Displaying the Table Structure

□ Use DESCRIBE command

e.g:

`desc faculty;`

Name	Null	Type
-----	-----	-----
FACULTYID	NOT NULL	NUMBER
LNAME	NOT NULL	VARCHAR2(30)
FNAME	NOT NULL	VARCHAR2(20)
DEPT		VARCHAR2(5)
OFFICEID		NUMBER
PHONE		VARCHAR2(15)
EMAIL		VARCHAR2(75)
RANK		CHAR(4)
8 rows selected		

Quiz

□ Identify the SELECT statements that execute successfully.

1.

```
SELECT first_name, last_name, job_id, salary*12 as yearly salary  
FROM employees;
```
2.

```
SELECT first_name, last_name, job_id, salary*12 "yearly salary"  
FROM employees;
```
3.

```
SELECT first_name, last_name, job_id, salary*12 as "yearly salary"  
FROM employees;
```
4.

```
SELECT first_name || last_name name || 'q{'s total salary is}' || salary  
FROM employees;
```

Limiting Rows using a Selection

- Restrict the rows that are returned by using the WHERE clause:

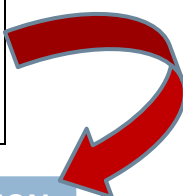
```
SELECT * | {[DISTINCT] column_name | expression [alias],.....}  
FROM table_name  
[WHERE conditon(s)];
```

- WHERE clause consists of three elements:
 - column name
 - comparison condition
 - column name, constant or list of values

Limiting Rows using a Selection *..cont.*

LAST_NAME	DEPT_ID	SALARY	COMMISSION	SALARY*COMMISSION
Siti Razilah	10	1000	(null)	(null)
Musa Ahmad	20	1200	0.2	240
Martin Luke	30	1500	0.6	900
Ammar Faizul	20	1800	0.3	540
Safiyah Shahizan	10	1230	(null)	(null)
Taylor Duff	20	2500	0.8	2000

```
SELECT last_name, dept_id, salary, commission  
FROM employee  
WHERE dept_id = 20;
```



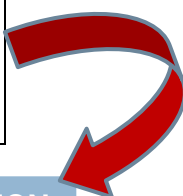
LAST_NAME	DEPT_ID	SALARY	COMMISSION
Musa Ahmad	20	1200	0.2
Ammar Faizul	20	1800	0.3
Taylor Duff	20	2500	0.8

Limiting Rows using a Selection *..cont.*

LAST_NAME	DEPT_ID	SALARY	COMMISSION	SALARY*COMMISSION
Siti Razilah	10	1000	(null)	(null)
Musa Ahmad	20	1200	0.2	240
Martin Luke	30	1500	0.6	900
Ammar Faizul	20	1800	0.3	540
Safiyah Shahizan	10	1230	(null)	(null)
Taylor Duff	20	2500	0.8	2000

1. Character strings and data values are enclosed with single quotation marks.
2. Character values are case-sensitive and date value are format-sensitive.

```
SELECT last_name, dept_id, salary, commission  
FROM employee  
WHERE last_name = 'Siti Razilah';
```



LAST_NAME	DEPT_ID	SALARY	COMMISSION
Siti Razilah	10	1000	(null)

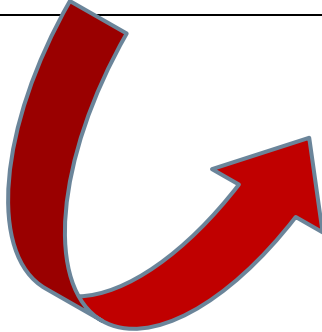
Comparison Operators

Operator	Meaning
=	Equal to
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
<>	Not equal to
BETWEEN...AND...	Between two values (inclusive)
IN(set)	Many any of a list of values
LIKE	Match a character pattern
IS NULL	Is a Null value

Limiting Rows using a Selection *..cont.*

LAST_NAME	DEPT_ID	SALARY	COMMISSION	SALARY*COMMISSION
Siti Razilah	10	1000	(null)	(null)
Musa Ahmad	20	1200	0.2	240
Martin Luke	30	1500	0.6	900
Ammar Faizul	20	1800	0.3	540
Safiyah Shahizan	10	1230	(null)	(null)
Taylor Duff	20	2500	0.8	2000

```
SELECT last_name, dept_id, salary, commission  
FROM employee  
WHERE dept_id between 10 and 20 ;
```

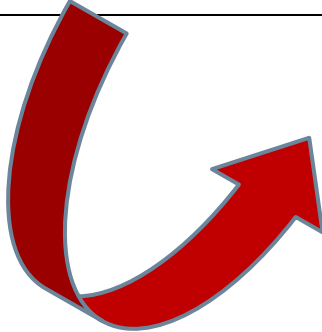


LAST_NAME	DEPT_ID	SALARY	COMMISSION
Siti Razilah	10	1000	(null)
Musa Ahmad	20	1200	0.2
Ammar Faizul	20	1800	0.3
Safiyah Shahizan	10	1230	(null)
Taylor Duff	20	2500	0.8

Limiting Rows using a Selection *..cont.*

LAST_NAME	DEPT_ID	SALARY	COMMISSION	SALARY*COMMISSION
Siti Razilah	10	1000	(null)	(null)
Musa Ahmad	20	1200	0.2	240
Martin Luke	30	1500	0.6	900
Ammar Faizul	20	1800	0.3	540
Safiyah Shahizan	10	1230	(null)	(null)
Taylor Duff	20	2500	0.8	2000

```
SELECT last_name, dept_id, salary, commission  
FROM employee  
WHERE last_name like 'M%';
```

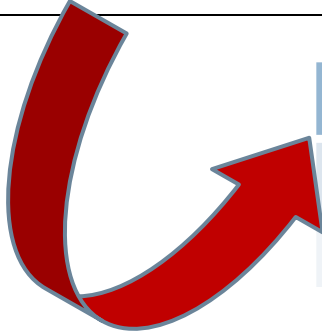


LAST_NAME	DEPT_ID	SALARY	COMMISSION
Musa Ahmad	20	1200	0.2
Martin Luke	30	1500	0.6

Limiting Rows using a Selection *..cont.*

LAST_NAME	DEPT_ID	SALARY	COMMISSION	SALARY*COMMISSION
Siti Razilah	10	1000	(null)	(null)
Musa Ahmad	20	1200	0.2	240
Martin Luke	30	1500	0.6	900
Ammar Faizul	20	1800	0.3	540
Safiyah Shahizan	10	1230	(null)	(null)
Taylor Duff	20	2500	0.8	2000

```
SELECT last_name, dept_id, salary, commission
FROM employee
WHERE last_name like '_a%' ;
```

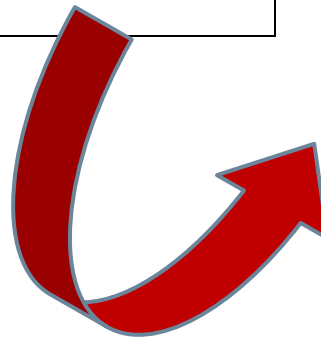


LAST_NAME	DEPT_ID	SALARY	COMMISSION
Martin Luke	30	1500	0.6
Safiyah Shahizan	10	1230	(null)

Limiting Rows using a Selection *..cont.*

LAST_NAME	DEPT_ID	SALARY	COMMISSION	SALARY*COMMISSION
Siti Razilah	10	1000	(null)	(null)
Musa Ahmad	20	1200	0.2	240
Martin Luke	30	1500	0.6	900
Ammar Faizul	20	1800	0.3	540
Safiyah Shahizan	10	1230	(null)	(null)
Taylor Duff	20	2500	0.8	2000

```
SELECT last_name, dept_id, salary, commission  
FROM employee  
WHERE dept_id in (10, 20) ;
```

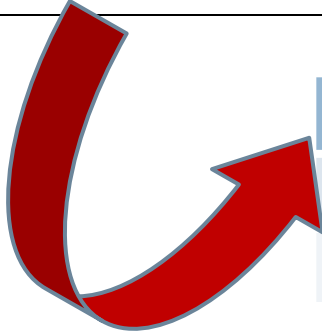


LAST_NAME	DEPT_ID	SALARY	COMMISSION
Siti Razilah	10	1000	(null)
Musa Ahmad	20	1200	0.2
Ammar Faizul	20	1800	0.3
Safiyah Shahizan	10	1230	(null)
Taylor Duff	20	2500	0.8

Limiting Rows using a Selection *..cont.*

LAST_NAME	DEPT_ID	SALARY	COMMISSION	SALARY*COMMISSION
Siti Razilah	10	1000	(null)	(null)
Musa Ahmad	20	1200	0.2	240
Martin Luke	30	1500	0.6	900
Ammar Faizul	20	1800	0.3	540
Safiyah Shahizan	10	1230	(null)	(null)
Taylor Duff	20	2500	0.8	2000

```
SELECT last_name, dept_id, salary, commission
FROM employee
WHERE commission is null;
```



LAST_NAME	DEPT_ID	SALARY	COMMISSION
Siti Razilah	10	1000	(null)
Safiyah Shahizan	10	1230	(null)

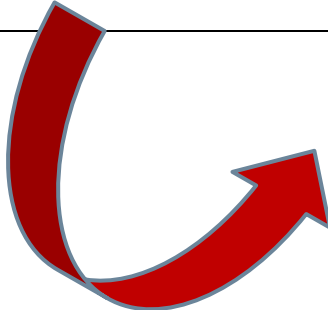
Conditions Using the Logical Operators

Operator	Meaning
AND	Return TRUE If both component conditions are true
OR	Return TRUE If either component condition is true
NOT	Return TRUE If the condition is true

AND Logical Operator

LAST_NAME	DEPT_ID	SALARY	COMMISSION	SALARY*COMMISSION
Siti Razilah	10	1000	(null)	(null)
Musa Ahmad	20	1200	0.2	240
Martin Luke	30	1500	0.6	900
Ammar Faizul	20	1800	0.3	540
Safiyah Shahizan	10	1230	(null)	(null)
Taylor Duff	20	2500	0.8	2000

```
SELECT last_name, dept_id, salary, commission
FROM employee
WHERE last_name like 'M%'
AND dept_id = 20;
```



LAST_NAME	DEPT_ID	SALARY	COMMISSION
Musa Ahmad	20	1200	0.2

AND Truth Table

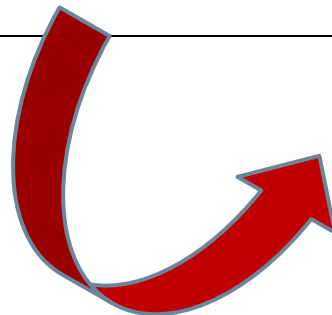
AND	TRUE	FALSE	NULL
TRUE	TRUE	FALSE	NULL
FALSE	FALSE	FALSE	FALSE
NULL	NULL	FALSE	NULL

OR Logical Operator

LAST_NAME	DEPT_ID	SALARY	COMMISSION	SALARY*COMMISSION
Siti Razilah	10	1000	(null)	(null)
Musa Ahmad	20	1200	0.2	240
Martin Luke	30	1500	0.6	900
Ammar Faizul	20	1800	0.3	540
Safiyah Shahizan	10	1230	(null)	(null)
Taylor Duff	20	2500	0.8	2000

```
SELECT last_name, dept_id, salary, commission
FROM employee
WHERE last_name like 'M%'
OR dept_id = 20;
```

LAST_NAME	DEPT_ID	SALARY	COMMISSION
Musa Ahmad	20	1200	0.2
Martin Luke	30	1500	0.6
Ammar Faizul	20	1800	0.3
Taylor Duff	20	2500	0.8



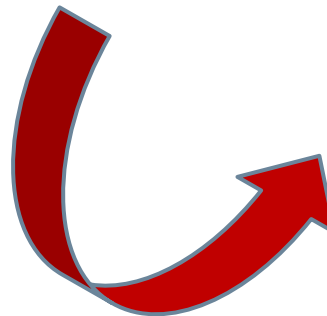
OR Truth Table

OR	TRUE	FALSE	NULL
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	NULL
NULL	TRUE	NULL	NULL

NOT Operator

LAST_NAME	DEPT_ID	SALARY	COMMISSION	SALARY*COMMISSION
Siti Razilah	10	1000	(null)	(null)
Musa Ahmad	20	1200	0.2	240
Martin Luke	30	1500	0.6	900
Ammar Faizul	20	1800	0.3	540
Safiyah Shahizan	10	1230	(null)	(null)
Taylor Duff	20	2500	0.8	2000

```
SELECT last_name, dept_id, salary, commission
FROM employee
WHERE dept_id NOT IN ('10','30');
```



LAST_NAME	DEPT_ID	SALARY	COMMISSION
Musa Ahmad	20	1200	0.2
Ammar Faizul	20	1800	0.3
Taylor Duff	20	2500	0.8

Sorting Rows

- Sort the retrieved rows with the ORDER BY clause:
 - ▣ ASC: Ascending order, default
 - ▣ DESC: Descending order

LAST_NAME	DEPT_ID	SALARY	COMMISSION	SALARY*COMMISSION
Siti Razilah	10	1000	(null)	(null)
Musa Ahmad	20	1200	0.2	240
Martin Luke	30	1500	0.6	900
Ammar Faizul	20	1800	0.3	540
Safiyah Shahizan	10	1230	(null)	(null)
Taylor Duff	20	2500	0.8	2000

```
SELECT last_name, dept_id, salary, commission
FROM employee
ORDER BY dept_id;
```



LAST_NAME	DEPT_ID	SALARY	COMMISSION
Siti Razilah	10	1000	(null)
Safiyah Shahizan	10	1230	(null)
Musa Ahmad	20	1200	0.2
Ammar Faizul	20	1800	0.3
Taylor Duff	20	2500	0.8
Martin Luke	30	1500	0.6

Sorting Rows ..cont.

- Sort the retrieved rows with the ORDER BY clause:
 - ▣ ASC: Ascending order, default
 - ▣ DESC: Descending order

LAST_NAME	DEPT_ID	SALARY	COMMISSION	SALARY*COMMISSION
Siti Razilah	10	1000	(null)	(null)
Musa Ahmad	20	1200	0.2	240
Martin Luke	30	1500	0.6	900
Ammar Faizul	20	1800	0.3	540
Safiyah Shahizan	10	1230	(null)	(null)
Taylor Duff	20	2500	0.8	2000

```
SELECT last_name, dept_id, salary, commission
FROM employee
ORDER BY dept_id DESC;
```



LAST_NAME	DEPT_ID	SALARY	COMMISSION
Martin Luke	30	1500	0.6
Taylor Duff	20	2500	0.8
Ammar Faizul	20	1800	0.3
Musa Ahmad	20	1200	0.2
Siti Razilah	10	1000	(null)
Safiyah Shahizan	10	1230	(null)

Sorting Rows ..cont.

- Sort the retrieved rows with the ORDER BY clause:
 - ▣ ASC: Ascending order, default
 - ▣ DESC: Descending order

LAST_NAME	DEPT_ID	SALARY	COMMISSION	SALARY*COMMISSION
Siti Razilah	10	1000	(null)	(null)
Musa Ahmad	20	1200	0.2	240
Martin Luke	30	1500	0.6	900
Ammar Faizul	20	1800	0.3	540
Safiyah Shahizan	10	1230	(null)	(null)
Taylor Duff	20	2500	0.8	2000

```
SELECT last_name, dept_id, salary, commission
FROM employee
ORDER BY 2 DESC
```



LAST_NAME	DEPT_ID	SALARY	COMMISSION
Martin Luke	30	1500	0.6
Taylor Duff	20	2500	0.8
Ammar Faizul	20	1800	0.3
Musa Ahmad	20	1200	0.2
Siti Razilah	10	1000	(null)
Safiyah Shahizan	10	1230	(null)

Sorting Rows ..cont.

- Sort the retrieved rows with the ORDER BY clause:
 - ▣ ASC: Ascending order, default
 - ▣ DESC: Descending order

LAST_NAME	DEPT_ID	SALARY	COMMISSION	SALARY*COMMISSION
Siti Razilah	10	1000	(null)	(null)
Musa Ahmad	20	1200	0.2	240
Martin Luke	30	1500	0.6	900
Ammar Faizul	20	1800	0.3	540
Safiyah Shahizan	10	1230	(null)	(null)
Taylor Duff	20	2500	0.8	2000

```
SELECT last_name, dept_id, salary, commission
FROM employee
ORDER BY dept_id, salary desc;
```

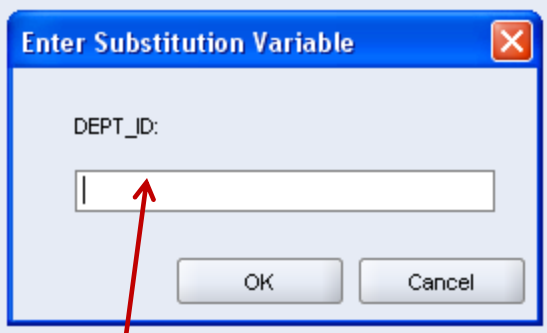


LAST_NAME	DEPT_ID	SALARY	COMMISSION
Martin Luke	30	1500	0.6
Taylor Duff	20	2500	0.8
Ammar Faizul	20	1800	0.3
Musa Ahmad	20	1200	0.2
Safiyah Shahizan	10	1230	(null)
Siti Razilah	10	1000	(null)

Substitution Variables

- Use a variable prefixed with an ampersand (&) to prompt the user for a value:

```
SELECT last_name, dept_id, salary, commission  
FROM employee  
WHERE dept_id = &dept_id;
```

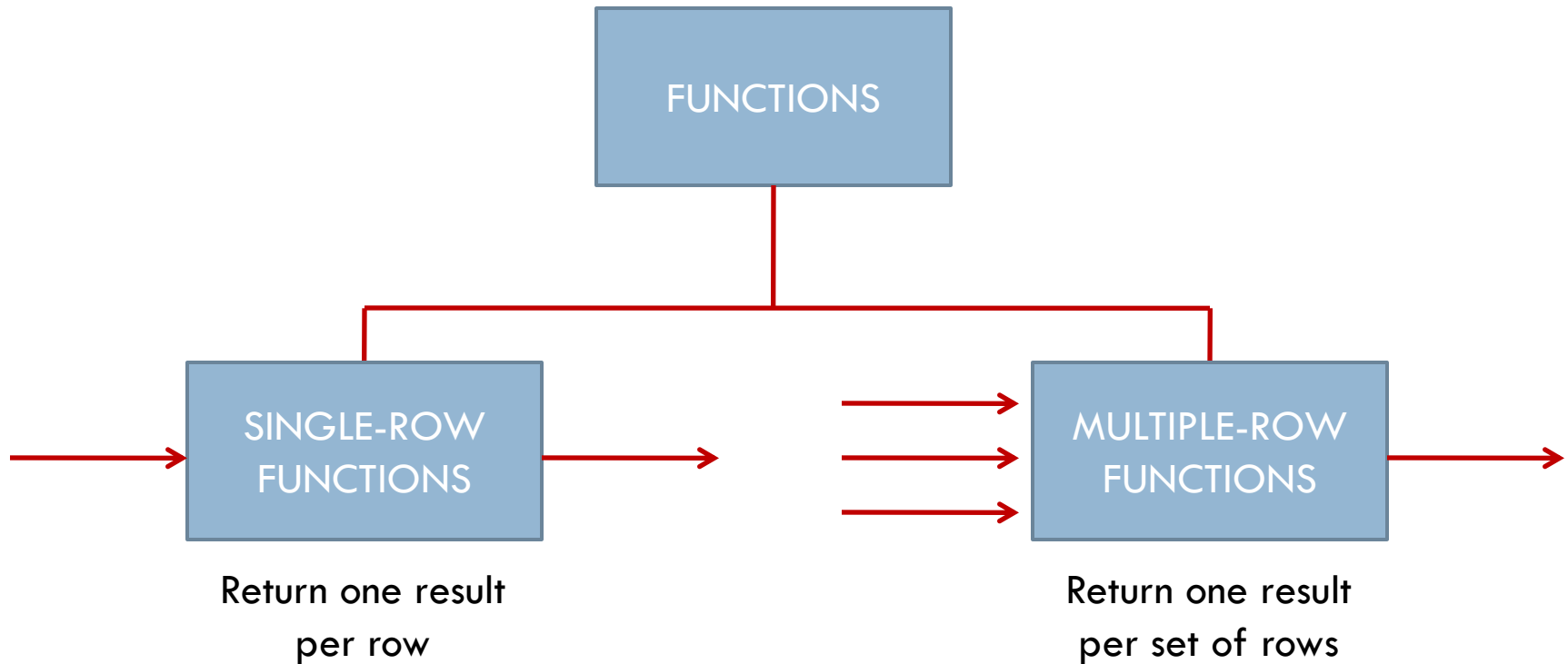


20



LAST_NAME	DEPT_ID	SALARY	COMMISSION
Musa Ahmad	20	1200	0.2
Ammar Faizul	20	1800	0.3
Taylor Duff	20	2500	0.8

Using Functions to Customize Output



Single Row Functions – Character Data Type

- ❑ LOWER(column/expression)
- ❑ UPPER(column/expression)
- ❑ INITCAP(column/expression)
- ❑ CONCAT(column/expression, column/expression)
- ❑ SUBSTR(column/expression, m[,n])
 - ❑ m – character position (if m is negative, the count starts from the end of the character value.)
 - ❑ n – character long (if n is omitted, all characters to the end of the string are returned)
- ❑ LENGTH(column/expression)
- ❑ INSTR(column/expression, 'string' [, m] , [n])

An Example

FUNCTION	RESULT
LOWER('Database application')	database application
UPPER('Database application')	DATABASE APPLICATION
INITCAP('Database application')	Database Application
CONCAT('Hello', 'World')	HelloWorld
SUBSTR('HelloWorld',2,5)	elloW
INSTR('HelloWorld', 'W')	6
LPAD(salary,10,'*')	*****24000
RPAD(salary,10,'*')	24000*****
REPLACE('JACK AND JUE', 'J', 'BL')	BLACK AND BLUE

Single Row Functions – Number Data Type

- ❑ **ROUND:** Rounds value to a special decimal
- ❑ **TRUNC:** Truncates value to specified decimal
- ❑ **MOD:** Returns remainder of division

FUNCTION	RESULT
ROUND(45.926, 2)	45.93
TRUNC(45.926, 2)	45.92
MOD(1600, 300)	100

Nesting Functions

LAST_NAME	DEPT_ID	SALARY	COMMISSION	SALARY*COMMISSION
Siti Razilah	10	1000	(null)	(null)
Musa Ahmad	20	1200	0.2	240
Martin Luke	30	1500	0.6	900
Ammar Faizul	20	1800	0.3	540
Safiyah Shahizan	10	1230	(null)	(null)
Taylor Duff	20	2500	0.8	2000

```
SELECT last_name,  
       UPPER(CONCAT(SUBSTR(last_name,1,8), '_US')) name  
FROM employee;
```

LAST_NAME	NAME
Siti Razilah	SITI RAZ_US
Musa Ahmad	MUSA AHM_US
Martin Luke	MARTIN L_US
Ammar Faizul	AMMAR FA_US
Safiyah Shahizan	SAFIYAH_US
Taylor Duff	TAYLOR D_US

NVL FUNCTION

- Convert a null value to an actual value:
 - ▣ Data types that can be used are date, character and number.
 - ▣ Data types must match:
 - NVL(commission, 0)
 - NVL(hire_date, '01-JAN-10')
 - NVL(job, 'No Job Yet')

CASE EXPRESSION

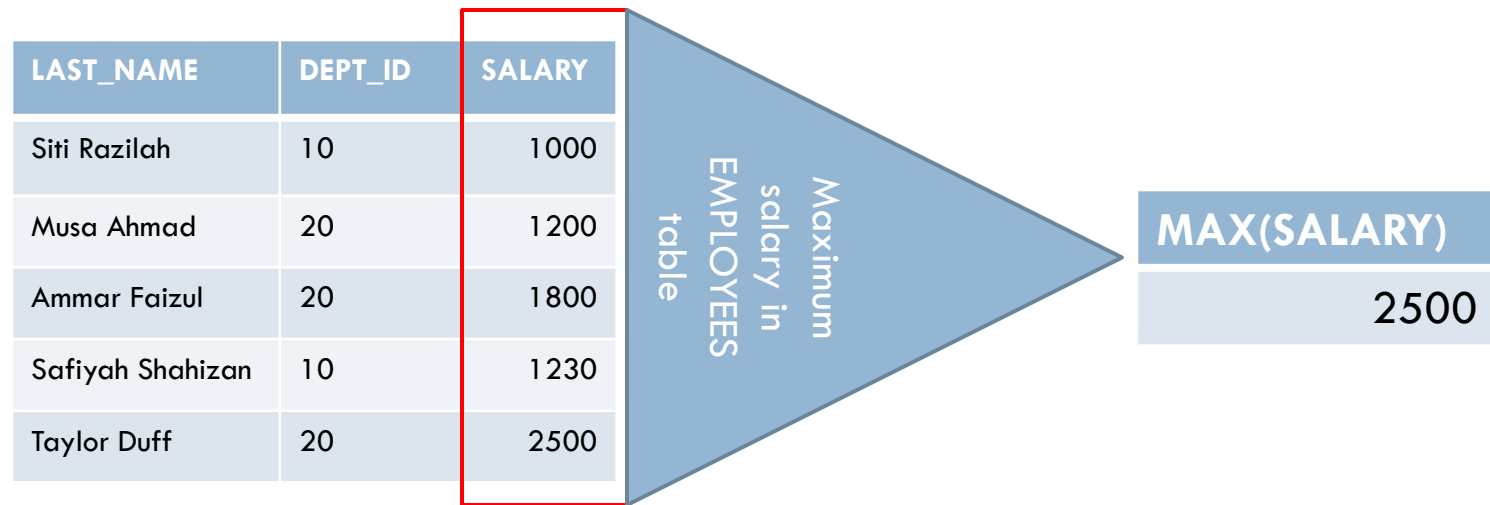
Not available in Microsoft Access. It is in SQL Server and Oracle.

```
Select AnimalID,  
    CASE  
        WHEN Date()-DateBorn < 90 Then 'Baby'  
        WHEN Date()-DateBorn >= 90  
            AND Date()-DateBorn < 270 Then 'Young'  
        WHEN Date()-DateBorn >= 270  
            AND Date()-DateBorn < 365 Then 'Grown'  
        ELSE 'Experienced'  
    END "STATUS"  
FROM Animal;
```

- Used to change data to a different context.
- Example: Define age categories for the animals.
 - Less than 3 months
 - Between 3 months and 9 months
 - Between 9 months and 1 year
 - Over 1 year

Multiple-Row Functions

- Group functions operate on sets of rows to give one result per group.



Type of Multiple Row Functions

- AVG(column)
- MAX(column)
- MIN(column)
- SUM(column)
- COUNT(*) or COUNT(column) or COUNT(DISTINCT column)
 - ▣ COUNT(*) returns the number of rows in a table.
 - ▣ COUNT(column) returns the number of rows with non-null values.
 - ▣ COUNT(DISTINCT column) returns the number of distinct column values.

Multiple-Row Functions and Null Values

- Group functions ignore null values in the column:

```
SELECT AVG(commission)
FROM employee;
```

- The NVL function forces group functions to include null values:

```
SELECT AVG(NVL(commission,0))
FROM employee;
```

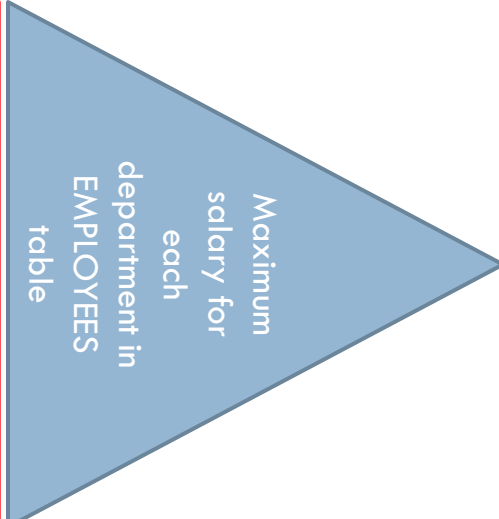
LAST_NAME	DEPT_ID	SALARY	COMMISSION
Siti Razilah	10	1000	(null)
Musa Ahmad	20	1200	0.2
Martin Luke	30	1500	0.6
Ammar Faizul	20	1800	0.3
Safiyah Shahizan	10	1230	(null)
Taylor Duff	20	2500	0.8



GROUP BY clause

- Group functions operate on sets of rows to give one result per group.

LAST_NAME	DEPT_ID	SALARY
Siti Razilah	10	1000
Musa Ahmad	20	1200
Martin Luke	30	1500
Ammar Faizul	20	1800
Safiyah Shahizan	10	1230
Taylor Duff	20	2500



DEPT_ID	MAX(SALARY)
10	1230
20	2500
30	1500

Creating Groups of Data

- To divide rows in a table into smaller groups by using GROUP BY clause.

```
SELECT * | {[DISTINCT] column_name | expression [alias],.....}  
FROM table_name  
[WHERE conditon(s)];  
[GROUP BY group_by_expression]  
[ORDER BY column];
```

Using the GROUP BY clause

- All the columns in the SELECT list that are not in group functions must be in the GROUP BY clause.
- However, the GROUP BY column does not have to be in the SELECT list.

```
SELECT dept_id, MAX(salary)
FROM employee
GROUP BY dept_id;
```

DEPT_ID	MAX(SALARY)
10	1230
20	2500
30	1500

```
SELECT MAX(salary)
FROM employee
GROUP BY dept_id;
```

MAX(SALARY)
1230
2500
1500

Use Group Functions in ORDER BY

```
SELECT dept_id, MAX(salary)
FROM employee
GROUP BY dept_id
ORDER BY MAX(salary);
```

DEPT_ID	MAX(SALARY)
20	2500
30	1500
10	1230

Restricting Group Results

LAST_NAME	DEPT_ID	SALARY
Siti Razilah	10	1000
Musa Ahmad	20	1200
Martin Luke	30	1500
Ammar Faizul	20	1800
Safiyah Shahizan	10	1230
Taylor Duff	20	2500

Maximum salary for each department BUT only for those whose average salary is greater than 1300

DEPT_ID	MAX(SALARY)
20	2500
30	1500

Restricting Group with HAVING clause

```
SELECT * | {[DISTINCT] column_name | expression [alias],.....}  
FROM table_name  
[WHERE conditon(s)];  
[GROUP BY group_by_expression]  
[HAVING group_coundition]  
[ORDER BY column];
```

```
SELECT dept_id, MAX(salary)  
FROM employee  
GROUP BY dept_id  
HAVING MAX(salary) > 1300;
```

End with... displaying data from a table

Then, we continue with displaying data from multiple tables or known as join tables.

Shall we start?

EMPLOYEE

EMPID	LAST_NAME	DEPT_ID	SALARY	COMMISSION
1	Siti Razilah	10	1000	(null)
2	Musa Ahmad	20	1200	0.2
3	Martin Luke	30	1500	0.6
4	Ammar Faizul	20	1800	0.3
5	Safiyah Shahizan	10	1230	(null)
6	Taylor Duff	20	2500	0.8

DEPENDENT

DEPID	NAME	DOB	STATUS	EMPID
101	Muhammad	12-10-1960	Suami	1
102	Sarah	03-03-1990	Anak	1
103	Linda	12-06-1993	Anak	1
104	Ali	27-09-1997	Anak	1
105	Sarimah	11-08-1976	Isteri	2
106	Adibah	24-07-2003	Anak	2
107	Haziq	06-03-2005	Anak	2
108	Elizabeth	04-04-1973	Isteri	6
109	Ridzuan	09-02-1990	Anak	4

LAST_NAME	SALARY	NAME	STATUS
Siti Razilah	1000	Muhammad	Suami
Siti Razilah	1000	Sarah	Anak
Siti Razilah	1000	Linda	Anak
Siti Razilah	1000	Ali	Anak
Musa Ahmad	1200	Sarimah	Isteri
Musa Ahmad	1200	Adibah	Anak
Musa Ahmad	1200	Haziq	Anak
Ammar Faizul	1800	Ridzuan	Anak
Taylor Duff	2500	Elizabeth	Isteri



Table Joins

- Process of combining data from two or more tables, normally using primary and/or foreign keys.
- Basic types of joins:
 - ▣ Natural Joins:
 - NATURAL JOIN clause
 - USING clause
 - ON clause
 - ▣ Self-Joins
 - ▣ Nonequijoins
 - ▣ Outer Joins:
 - LEFT OUTER JOIN clause
 - RIGHT OUTER JOIN clause
 - FULL OUTER JOIN clause
 - ▣ CROSS JOIN clause

Joining Tables Using SQL:1999 syntax

```
SELECT table1.column, table2.column  
FROM table1  
[NATURAL JOIN table2] |  
[JOIN table2 USING (column_name)] |  
[JOIN table2 on (table1.column_name = table2.column_name)] |  
[LEFT | RIGHT | FULL OUTER JOIN table2 on table1.column_name =  
table2.column_name)] |  
[CROSS JOIN table2];
```

Creating NATURAL JOIN clause

- The NATURAL JOIN clause is based on all the columns in the two tables that have the same name.
- It selects rows from the two tables that have equal values in all matched columns.
- If the columns having same name but different data type, an error is returned.

Example of Using NATURAL JOIN clause

EMPLOYEE

EMPID	LAST_NAME	DEPT_ID	SALARY	COMMISSION
1	Siti Razilah	10	1000	(null)
2	Musa Ahmad	20	1200	0.2
3	Martin Luke	30	1500	0.6
4	Ammar Faizul	20	1800	0.3
5	Safiyah Shahizan	10	1230	(null)
6	Taylor Duff	20	2500	0.8

DEPARTMENT

DEPT_ID	DEPT_NAME
10	Department of Networking
20	Department of Info. Science
30	Department of Multimedia
40	Department of Computer Science
50	Department of Applied Science
60	Department of Robotics

SELECT last_name, salary, dept_name
FROM employee
NATURAL JOIN department



LAST_NAME	SALARY	DEPT_NAME
Siti Razilah	1000	Department of Networking
Musa Ahmad	1200	Department of Info. Science
Martin Luke	1500	Department of Multimedia
Ammar Faizul	1800	Department of Info. Science
Safiyah Shahizan	1230	Department of Networking
Taylor Duff	2500	Department of Info. Science

Creating join with the USING clause

- If several columns have same name but different data type then use the USING clause to specify the columns for the equijoin.
- Or, use the USING clause when wanted to match only one column of many column matches.

Example of Using NATURAL JOIN clause

EMPLOYEE

EMPID	LAST_NAME	DEPT_ID	SALARY	COMMISSION
1	Siti Razilah	10	1000	(null)
2	Musa Ahmad	20	1200	0.2
3	Martin Luke	30	1500	0.6
4	Ammar Faizul	20	1800	0.3
5	Safiyah Shahizan	10	1230	(null)
6	Taylor Duff	20	2500	0.8

DEPARTMENT

DEPT_ID	DEPT_NAME
10	Department of Networking
20	Department of Info. Science
30	Department of Multimedia
40	Department of Computer Science
50	Department of Applied Science
60	Department of Robotics

```
SELECT last_name, salary, dept_name  
FROM employee  
JOIN department  
USING (dept_id)
```



LAST_NAME	SALARY	DEPT_NAME
Siti Razilah	1000	Department of Networking
Musa Ahmad	1200	Department of Info. Science
Martin Luke	1500	Department of Multimedia
Ammar Faizul	1800	Department of Info. Science
Safiyah Shahizan	1230	Department of Networking
Taylor Duff	2500	Department of Info. Science

Using Table Aliases with the USING clause

- ❑ Do not qualify a column that is used in the USING clause.

```
SELECT e.last_name, e.salary, d.dept_name  
FROM employee e  
JOIN department d  
USING (dept_id)  
WHERE d.dept_id in ('10','20');
```



```
SELECT e.last_name, e.salary, d.dept_name  
FROM employee e  
JOIN department d  
USING (dept_id)  
WHERE dept_id in ('10','20');
```



Creating Joins with the ON clause

- ❑ NATURAL JOIN is basically equijoin of all columns with the same name.
- ❑ For arbitrary conditions (selected columns) use the ON clause
- ❑ Example:

```
SELECT e.last_name, e.salary, d.dept_name  
FROM employee e  
JOIN department d  
ON (e.dept_id = d.dept_id);
```

Applying Additional Conditions to a Join

- Use the AND clause or the WHERE clause to specify additional conditions.
- Example:

```
SELECT e.last_name, e.salary, d.dept_name  
FROM employee e  
JOIN department d  
ON (e.dept_id = d.dept_id)  
AND e.salary > 1000;
```

```
SELECT e.last_name, e.salary, d.dept_name  
FROM employee e  
JOIN department d  
ON (e.dept_id = d.dept_id)  
WHERE e.salary > 1000;
```

SELF-JOIN

□ Joining a table to itself.

EMPLOYEE (WORKER)

EMPID	LAST_NAME	DEPT_ID	SALARY	COMMISSION	MGR_ID
1	Siti Razilah	10	1000	(null)	6
2	Musa Ahmad	20	1200	0.2	4
3	Martin Luke	30	1500	0.6	6
4	Ammar Faizul	20	1800	0.3	6
5	Safiyah Shahizan	10	1230	(null)	1
6	Taylor Duff	20	2500	0.8	(null)



EMPLOYEE (MANAGER)

EMPID	LAST_NAME	DEPT_ID	SALARY	COMMISSION	MGR_ID
1	Siti Razilah	10	1000	(null)	6
2	Musa Ahmad	20	1200	0.2	4
3	Martin Luke	30	1500	0.6	6
4	Ammar Faizul	20	1800	0.3	6
5	Safiyah Shahizan	10	1230	(null)	1
6	Taylor Duff	20	2500	0.8	(null)

Self-Join using the ON clause

```
SELECT worker.last_name emp, worker.dept_id department,  
       worker.salary salary, manager.last_name manager  
FROM employee worker  
JOIN employee manager  
ON (worker.mgr_id = manager.empid);
```

EMP	DEPARTMENT	SALARY	MANAGER
Safiyah Shahizan	10	1230	Siti Razilah
Musa Ahmad	20	1200	Ammar Faizal
Ammar Faizul	20	1800	Taylor Duff
Siti Razilah	10	1000	Taylor Duff
Martin Luke	30	1500	Taylor Duff

Nonequijoins

- A join condition containing something other than an equality operator.

EMPLOYEE

EMPID	LAST_NAME	DEPT_ID	SALARY	COMMISSION	MGR_ID
1	Siti Razilah	10	1000	(null)	6
2	Musa Ahmad	20	1200	0.2	4
3	Martin Luke	30	1500	0.6	6
4	Ammar Faizul	20	1800	0.3	6
5	Safiyah Shahizan	10	1230	(null)	1
6	Taylor Duff	20	2500	0.8	(null)

JOB_GRADE

GRADE_LEVEL	LOW_SAL	MAX_SAL
A	0	999
B	1000	1999
C	2000	2999
D	3000	3999
E	4000	10000

```
SELECT e.last_name, e.salary, j.grade_level
FROM employee e
JOIN job_grade j
ON e.salary
   BETWEEN j.low_sal AND j.max_sal
```

LAST_NAME	SALARY	GRADE_LEVEL
Siti Razilah	1000	B
Musa Ahmad	1200	B
Martin Luke	1500	B
Ammar Faizul	1800	B
Safiyah Shahizan	1230	B
Taylor Duff	2500	C

Nonequijoins ..cont.

- AccountsReceivable
- Categorize by Days Late
 - 30, 90, 120+
- Three queries?
- New table for business rules

AR(TransactionID, CustomerID, Amount, DateDue)

LateCategory(Category, MinDays, MaxDays, Charge, ...)

Month	30	90	3%
Quarter	90	120	5%
Overdue	120	9999	10%

```
SELECT *  
FROM AR JOIN LateCategory  
ON ((Date() - AR.DateDue) >= LateCategory.MinDays)  
AND ((Date() - AR.DateDue) < LateCategory.MaxDays)
```

Returning Record with No Direct Match

- Use OUTER joins.
 - ▣ LEFT OUTER JOIN clause
 - ▣ RIGHT OUTER JOIN clause
 - ▣ FULL OUTER JOIN clause

LEFT OUTER JOIN

- Used when you may not have matching data in the secondary table, but still want to include the data you have in the primary table.

NOTE: The primary table is normally listed on the left side of the equation, the secondary on the right side.

An Example of Left Joins

Insert a new department in table dept (for example, dept_id=50).

```
Select d.dept_id, d.dept_name, e.dept_id, e.last_name  
FROM dept d LEFT OUTER JOIN employee e  
ON d.dept_id = e.dept_id;
```

DEPT_ID	DEPT_NAME	DEPT_ID	LAST_NAME
10	Department of Networking	10	Siti Razilah
20	Department of Info. Science	20	Musa Ahmad
30	Department of Multimedia	30	Martin Luke
20	Department of Info. Science	20	Ammar Faizul
10	Department of Networking	10	Safiyah Shahizan
20	Department of Info. Science	20	Taylor Duff
50	Department of Math		

RIGHT OUTER JOINS

- Used when you may have data in the secondary table with no matching data in the primary table.
- Example:

Insert a new employee in table employee (for example, empid=7).

```
Select d.dept_id, d.dept_name, e.dept_id, e.last_name  
FROM dept d RIGHT OUTER JOIN employee e  
ON d.dept_id = e.dept_id;
```

RIGHT OUTER JOINS *..cont.*

Insert a new employee in table employee (for example, empid=7).

```
Select d.dept_id, d.dept_name, e.dept_id, e.last_name  
FROM dept d RIGHT OUTER JOIN employee e  
ON d.dept_id = e.dept_id;
```

DEPT_ID	DEPT_NAME	DEPT_ID	LAST_NAME
			Gopal
10	Department of Networking	10	Siti Razilah
20	Department of Info. Science	20	Musa Ahmad
30	Department of Multimedia	30	Martin Luke
20	Department of Info. Science	20	Ammar Faizul
10	Department of Networking	10	Safiyah Shahizan
20	Department of Info. Science	20	Taylor Duff

FULL OUTER JOIN

```
Select d.dept_id, d.dept_name, e.dept_id, e.last_name  
FROM dept d RIGHT OUTER JOIN employee e  
ON d.dept_id = e.dept_id;
```

DEPT_ID	DEPT_NAME	DEPT_ID	LAST_NAME
			Gopal
10	Department of Networking	10	Siti Razilah
20	Department of Info. Science	20	Musa Ahmad
30	Department of Multimedia	30	Martin Luke
20	Department of Info. Science	20	Ammar Faizul
10	Department of Networking	10	Safiyah Shahizan
20	Department of Info. Science	20	Taylor Duff
50	Department of Math		

Generating a Cartesian Product

□ Use CROSS JOIN clause

EMPLOYEE (6 rows)

LAST_NAME	DEPT_ID	SALARY	COMMISSION
Siti Razilah	10	1000	(null)
Musa Ahmad	20	1200	0.2
Martin Luke	30	1500	0.6
Ammar Faizul	20	1800	0.3
Safiyah Shahizan	10	1230	(null)
Taylor Duff	20	2500	0.8

DEPART (4 rows)

DEPT_ID	DEPT_NAME
10	Department of Networking
20	Department of Info. Science
30	Department of Multimedia
40	Department of Computer Science

Cartesian Product: $6 \times 4 = 24$ rows

Generating a Cartesian Product ..cont.

DEPT_ID	DEPT_NAME	DEPT_ID	LNAME
10	Department of Networking	10	Siti Razilah
10	Department of Networking	20	Musa Ahmad
10	Department of Networking	30	Martin Luke
10	Department of Networking	20	Ammar Faizul
10	Department of Networking	10	Safiyah Sahizan
10	Department of Networking	20	Taylor Duff
20	Department of Info. Science	10	Siti Razilah
20	Department of Info. Science	20	Musa Ahmad
20	Department of Info. Science	30	Martin Luke
20	Department of Info. Science	20	Ammar Faizul
20	Department of Info. Science	10	Safiyah Sahizan
20	Department of Info. Science	20	Taylor Duff
30	Department of Multimedia	10	Siti Razilah
30	Department of Multimedia	20	Musa Ahmad
30	Department of Multimedia	30	Martin Luke
30	Department of Multimedia	20	Ammar Faizul
30	Department of Multimedia	10	Safiyah Sahizan
30	Department of Multimedia	20	Taylor Duff
40	Department of Math	10	Siti Razilah
40	Department of Math	20	Musa Ahmad
40	Department of Math	30	Martin Luke
40	Department of Math	20	Ammar Faizul
40	Department of Math	10	Safiyah Sahizan
40	Department of Math	20	Taylor Duff

24 rows selected

Using Subqueries to Solve Queries

Main Query:

Which employees have salaries greater than Ammar Faizul's salary?



Sub Query:

How much is Ammar Faizul's salary?



SubQuery Syntax

```
SELECT select_list
FROM table
WHERE expr operator
           (Select select_list
            FROM TABLE);
```

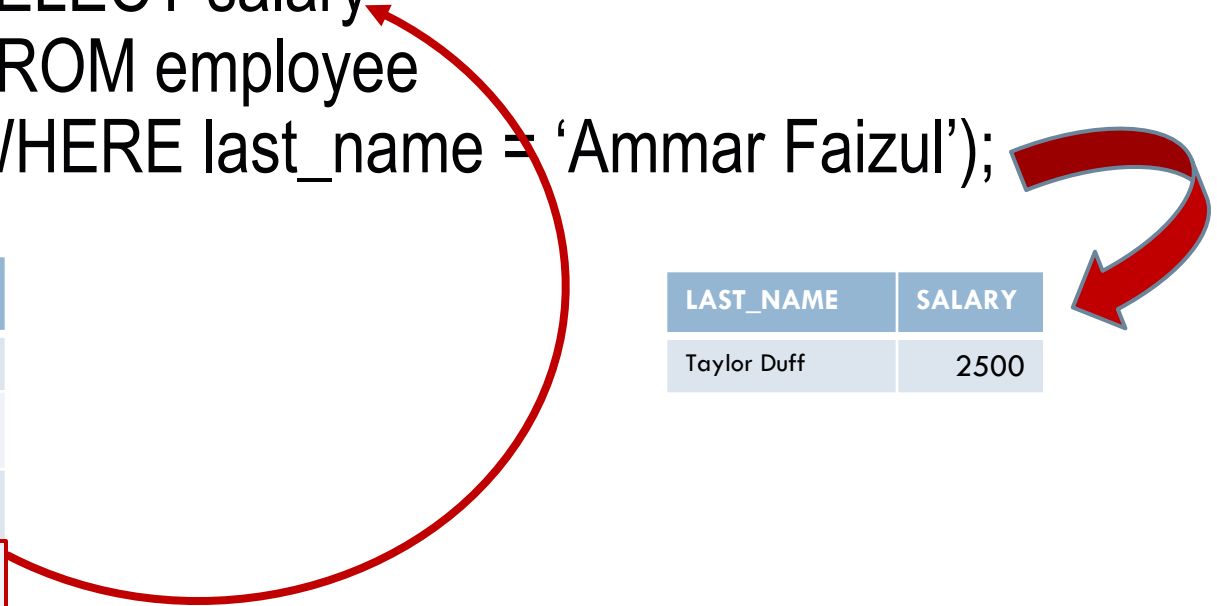
- The subquery (inner query) executes before the main query (outer query)
- The result of subquery is used by the main query.

An Example of Subquery

```
SELECT last_name, salary
FROM employee
where salary > (SELECT salary
                 FROM employee
                 WHERE last_name = 'Ammar Faizul');
```

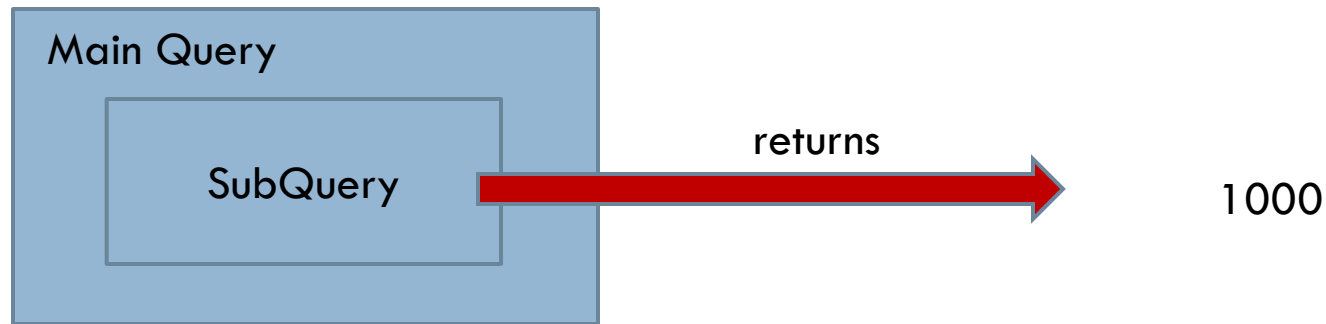
LAST_NAME	DEPT_ID	SALARY
Siti Razilah	10	1000
Musa Ahmad	20	1200
Martin Luke	30	1500
Ammar Faizul	20	1800
Safiyah Shahizan	10	1230
Taylor Duff	20	2500

LAST_NAME	SALARY
Taylor Duff	2500

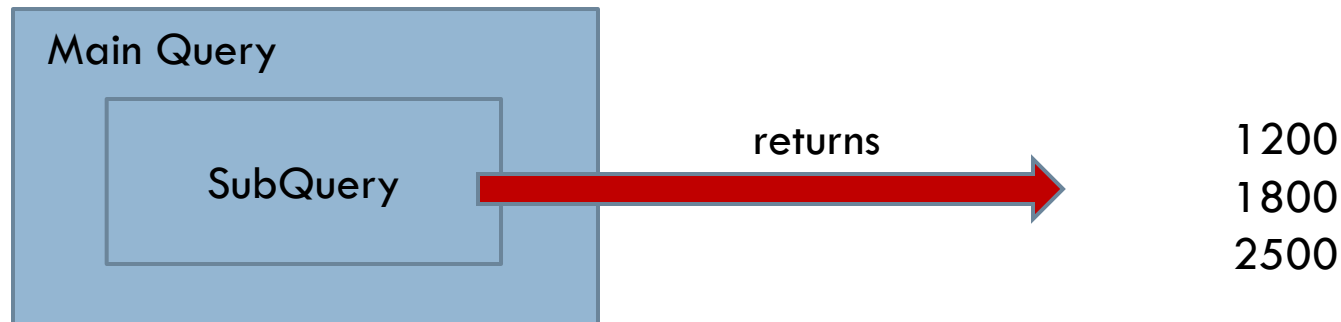


Types of Subqueries

□ Single-row subquery



□ Multiple-row subquery



An Example

```
SELECT last_name, salary
FROM employee
where salary = (SELECT salary
                FROM employee
                WHERE dept_id = 20);
```



Single-row operator with
multiple-row subquery

```
SELECT last_name, salary
FROM employee
where salary in (SELECT salary
                FROM employee
                WHERE dept_id = 20);
```

EMPID	LAST_NAME	DEPT_ID	SALARY	COMM
1	Siti Razilah	10	1000	
2	Musa Ahmad	20	1200	
3	Martin Luke	30	1500	
4	Ammar Faizul	20	1800	
5	Safiyah Shahizan	10	1230	
6	Taylor Duff	20	2500	



Subquery Characteristics

- ❑ Can be used in the SELECT, CREATE, INSERT, UPDATE and DELETE statements
- ❑ Can be used as part of a condition in the WHERE clause
- ❑ Can be used in multiple AND or OR predicates of the same SQL statement
- ❑ Is enclosed in parenthesis and must appear on the right in a WHERE clause condition
- ❑ Cannot include an ORDER BY clause
- ❑ The number of rows returned by the subquery must match the number expected by the main query

Combining Subqueries

- Multiple subqueries can be used to check for more than one condition in a statement.
- Example:

```
SELECT last_name, salary
FROM employees
WHERE dept_id = (SELECT dept_id
                  FROM employee
                  WHERE last_name = 'Taylor Duff')
AND salary > (SELECT salary
               FROM employee
               WHERE last_name = 'Taylor Duff');
```
- Same or different types can be nested.

NOTE: Nested subqueries are executed from the most deeply nested to the least deeply nested subquery.

Example

Show all customers who have placed an order

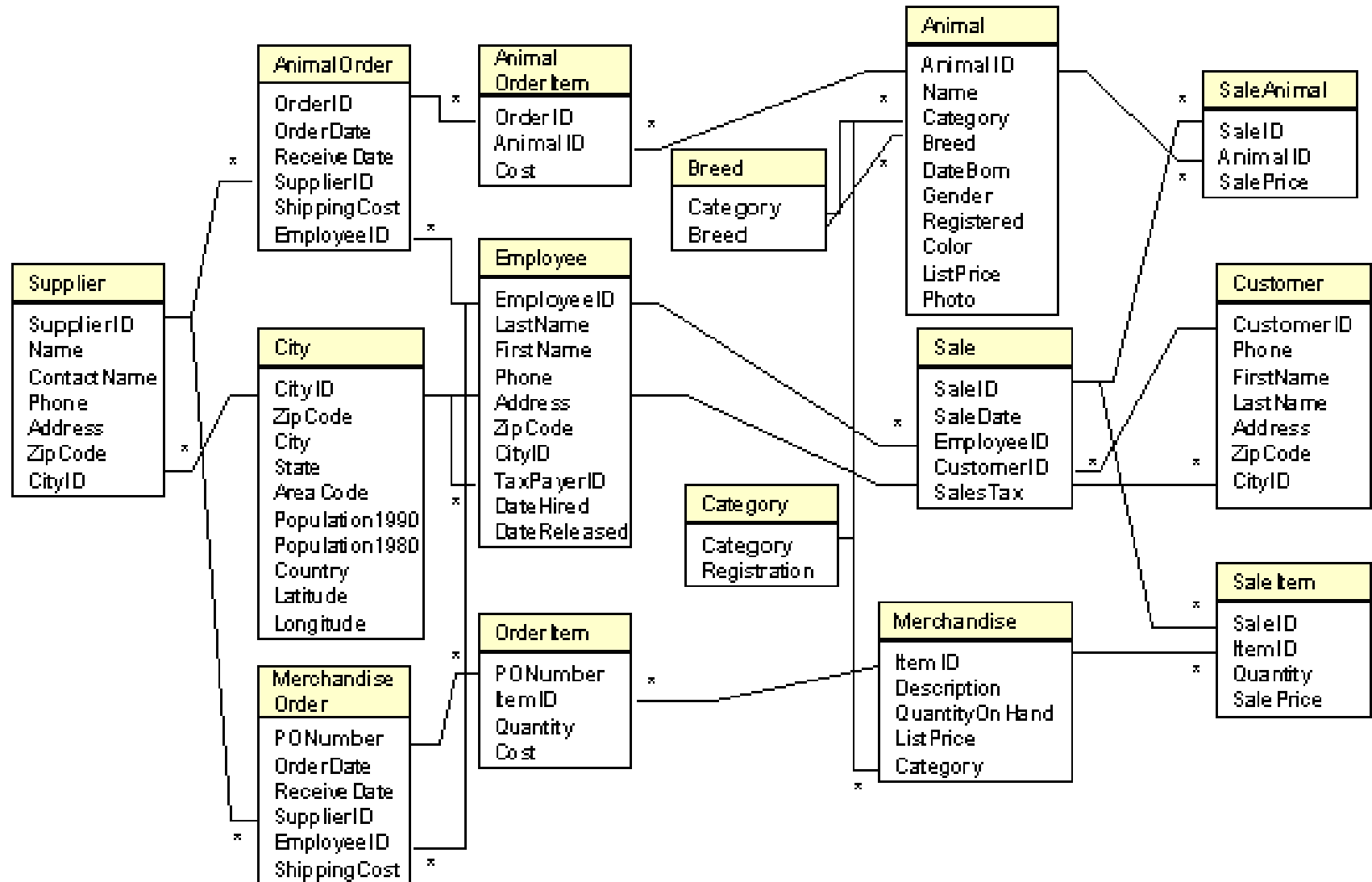
```
SELECT customer_name FROM customer_t  
WHERE customer_ID IN
```

```
(SELECT DISTINCT customer_ID FROM order_t);
```



Subquery is embedded in parentheses. In this case it returns a list that will be used in the WHERE clause of the outer query

Sally Pet Store

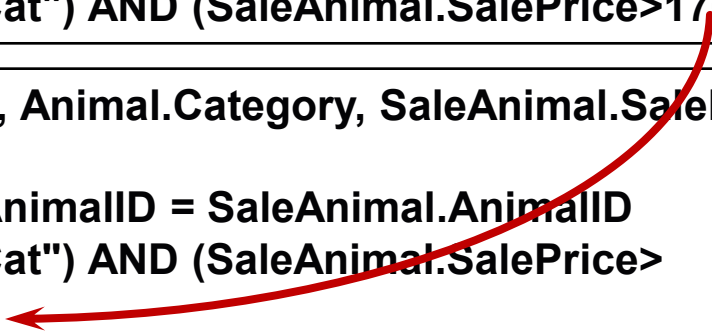


Subquery for Calculation

- Which cats sold for more than the average sale price of cats?
 - Assume we know the average price is \$170.
 - Usually we need to compute it first.

```
SELECT SaleAnimal.AnimalID, Animal.Category,  
SaleAnimal.SalePrice  
FROM Animal  
SaleAnimal ON Animal.AnimalID = SaleAnimal.AnimalID  
WHERE ((Animal.Category="Cat") AND (SaleAnimal.SalePrice>170));
```

```
SELECT SaleAnimal.AnimalID, Animal.Category, SaleAnimal.SalePrice  
FROM Animal  
JOIN SaleAnimal ON Animal.AnimalID = SaleAnimal.AnimalID  
WHERE ((Animal.Category="Cat") AND (SaleAnimal.SalePrice>  
    ( SELECT AVG(SalePrice)  
      FROM Animal  
      JOIN SaleAnimal ON Animal.AnimalID = SaleAnimal.AnimalID  
      WHERE (Animal.Category="Cat")  
    )  
    ) );
```

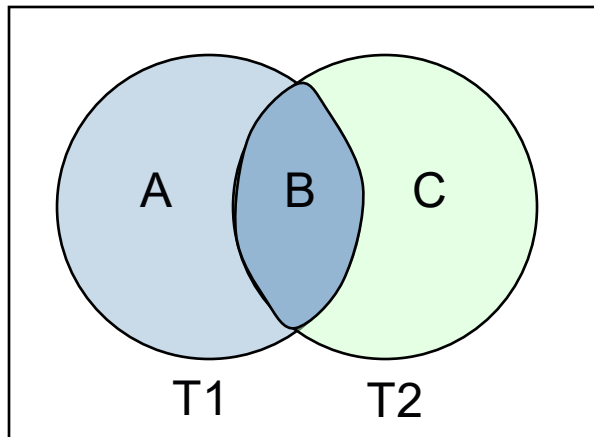


Using NOT IN with a Subquery

- ☛ Which animals have **not** been sold?
 - Start with list of all animals.
 - Subtract out list of those who were sold.

```
SELECT Animal.AnimalID, Animal.Name, Animal.Category
FROM Animal
WHERE (Animal.AnimalID Not In
      (SELECT AnimalID From SaleAnimal));
```

UNION, INTERSECT, MINUS



List the name of any employee who has worked for both the East and West regions.

T1 UNION T2	A + B + C
T1 INTERSECT T2	B
T1 EXCEPT T2	A

```
SELECT EID, Name
FROM EmployeeEast
INTERSECT
SELECT EID, Name
FROM EmployeeWest
```

UNION

```
SELECT EID, Name, Phone, Salary, 'East' AS Office  
FROM EmployeeEast
```

UNION

```
SELECT EID, Name, Phone, Salary, 'West' AS Office  
FROM EmployeeWest
```

EID	Name	Phone	Salary	Office
352	Jones	3352	45,000	East
876	Inez	8736	47,000	East
372	Stoiko	7632	38,000	East
890	Smythe	9803	62,000	West
361	Kim	7736	73,000	West

- ☛ Offices in Los Angeles and New York.
- ☛ Each has an Employee table (East and West).
- ☛ Need to search data from both tables.
- ☛ Columns in the two SELECT lines must match the column number and column type.

Summary

- In this chapter you learned how to:
 - ▣ Define key terms: join, equijoin, self-join, nonequijoin, natural join, outer join, set operators and subquery.
 - ▣ Apply the DML command to insert, update and delete data from table.
 - ▣ Write single and multiple table queries using SQL commands
 - ▣ Write queries using subqueries.